

Strukturalni paterni

1.Adapter pattern

Adapter patern je strukturni dizajnerski obrazac koji omogućava komunikaciju između klasa sa nekompatibilnim interfejsima.

Adapter djeluje kao posrednik koji prevodi pozive iz jednog interfejsa u drugi koji sistem može koristiti. Klasa NotifikacijaServis predstavlja eksterni servis za slanje različitih tipova notifikacija (SMS, Email, Push). NotifikacijaAdapter prilagođava metode tog servisa interfejsu INotifikacija koji koristi ostatak sistema.

2.Facade pattern

Facade pattern je korišten za pojednostavljenje rada sa sistemom parking rezervacija.

Klasa ParkingFacade pruža jedinstven interfejs za složene operacije kao što su rezervacija mjesta, generisanje QR koda i slanje notifikacija, bez potrebe da korisnik direktno komunicira sa više podsistema.

3.Decorator pattern

Dinamički dodaje nove funkcionalnosti objektima bez izmjene njihove osnovne klase. Moglo bi se dodati kod rezervacija ili parking mjesta. Na primjer: VIPParkingDecorator, NatkrivenoMjestoDecorator , PunjačZaEVDecorator dodaju dodatne opcije parking mjestu.

4. Composite pattern

Omogućava tretiranje pojedinačnih objekata i grupa objekata na isti način. Moglo bi se dodati kod organizacije parkinga.

Na primjer: Parking, Sprat, Zona, ParkingMjesto. Sve može implementirati zajednički interfejs.

5. Bridge pattern

Razdvaja apstrakciju od implementacije kako bi se mogle nezavisno mijenjati. U našem slučaju nije baš potrebno.

6. Proxy pattern

Kontroliše pristup drugom objektu. Može se dodati kod administratorskih funkcija. Na primjer: ParkingProxy provjerava da li korisnik ima admin prava prije izmjena parkinga.